

DATABASE ENCRYPTION

Encryption at Column Level

1. Configure the Cryptographic Cipher

Before creating tables or writing data, establish the mathematical rules for the encryption engine.

- Select a strong, industry-standard algorithm such as AES-256-CBC or AES-GCM.

```
SET @@block_encryption_mode = 'aes-256-cbc';
```

2. Design the Database Schema

You must prepare your destination table to handle binary data payloads instead of standard text strings.

- Convert Text Columns: Change target columns from standard text types (VARCHAR, TEXT) to binary storage types (VARBINARY, BLOB).
- Add an IV Column: Allocate a dedicated binary column (such as VARBINARY(16)) to preserve the unique Initialization Vector (IV) generated for each individual record.
- Optional Lookup Column: If you need to search or index the column without decrypting it, create a separate column to store a one-way cryptographic hash (e.g., SHA-256) of the plaintext.

3. Generate a Secure Key Management Strategy

An encryption algorithm is only as secure as the key that locks it.

- Define a strong secret key (typically a 32-byte string for AES-256).
- Decide how your environment will pass this key to the query engine. In production environments, keys should live in an isolated application configuration file or a dedicated Key Management Service (KMS) rather than being hardcoded into database scripts, triggers, or stored procedures.

4. Execute the Encryption and Write Process

When writing new records or updating existing entries, your SQL queries must route the data through the database engine's cryptographic wrapper.

- Generate a fresh IV: Use a random byte generator function to create a new, unpredictable IV for the execution row.
- Scramble the Data: Call your database's encryption function (such as AES_ENCRYPT), passing the plaintext string, the secret encryption key, and the fresh IV.
- Commit to Disk: Write both the resulting encrypted binary payload and the raw IV together into the row.

5. Execute the Read and Decryption Process

To pull information back out of the database for user consumption, reverse the mathematical pipeline during selection operations.

- Target the Row: Query the table utilizing standard filters or your indexed lookup hash.

- Unscramble the Data: Call your database's decryption function (such as AES_DECRYPT), passing the encrypted binary column, the matching secret key, and the stored row-specific IV column.
- Cast back to Text: Explicitly convert the output bytes back into a standard text characters (using functions like CAST(... AS CHAR)) so your application layer can easily read it.

SQL Program

```
CREATE DATABASE powerm;
USE powerm;
```

STEP 1 SET @@block_encryption_mode = 'aes-256-cbc';

```
CREATE TABLE client_billing (
  client_id INT AUTO_INCREMENT PRIMARY KEY,
  cardholder_name VARCHAR(100) NOT NULL,
  -- Holds the raw encrypted binary string
  card_number_encrypted VARBINARY(256) NOT NULL,
  -- Holds the 16-byte initialization vector
  encryption_iv VARBINARY(16) NOT NULL,
  -- One-way hash allowing fast, indexed WHERE lookups
  card_token_hash CHAR(64) NOT NULL,
  INDEX (card_token_hash)
);
```

STEP 3 SET @secret_key = 'DatabaseSecretEncryptionKey2026!';
SET @raw_card = '1110987654321';
SET @row_iv = RANDOM_BYTES(16);

```
INSERT INTO client_billing (cardholder_name, card_number_encrypted,
  encryption_iv, card_token_hash)
VALUES (
```

STEP 4 'John Doe',
AES_ENCRYPT(@raw_card, @secret_key, @row_iv),
@row_iv,
SHA2(@raw_card, 256) -- Generates an irreversible lookup string
);

Decryption:

```
SET @secret_key = 'DatabaseSecretEncryptionKey2026!';  
SET @search_card = '1110987654321';
```

```
SELECT  
    cardholder_name,  
    CAST(AES_DECRYPT(card_number_encrypted, @secret_key, encryption_iv) AS  
    CHAR) AS visible_card_number  
FROM client_billing  
-- High-performance indexed lookup  
WHERE card_token_hash = SHA2(@search_card, 256);
```